

Table des matières

1. Pour commencer
2. Création de processus
3. Les tubes anonymes
4. Les tubes nommés
5. Concurrency

1. Pour commencer

Le but de ce TP est de manipuler les appels système relatifs à la gestion des processus et aux tubes nommés et anonymes.

Pour l'ensemble des questions suivantes, vous devez utiliser les fonctions C qui ont été introduites dans les cours correspondants. À noter que la gestion d'erreurs doit être correctement mise en place pour tous les appels systèmes et l'utilisation d'un makefile est obligatoire.

2. Création de processus

L'objectif de cet exercice est de manipuler les commandes `fork`, `wait` (ou `waitpid` seulement si nécessaire) et `execve`.

1. Écrivez un programme qui prend en argument un entier `n` et qui crée ensuite `n` processus fils. Chaque fils affiche son numéro de création (de 1 à `n`) et un nombre aléatoire. Le père attend la fin de chaque fils et affiche au fur-et-à-mesure le PID du fils qui s'arrête.

Vous devez utiliser la procédure `srand` pour initialiser le générateur de nombres pseudo-aléatoires. Elle prend en paramètre une graine. Habituellement, on utilise le retour de la fonction `time`. La fonction `rand` permet ensuite de générer des entiers aléatoires dans l'intervalle `[0;RAND_MAX]`

2. Modifiez votre programme pour que chaque fils attende un temps aléatoire (vous pouvez utiliser `sleep`). À la fin de chaque fils, le père indique le numéro de création et le PID du fils qui s'est arrêté. Pour le numéro de création, utilisez la valeur de retour envoyée par `exit`.
3. Comment faire pour éviter d'utiliser la valeur retournée par `exit` ?

Indications : dans le père, la fonction `fork` retourne le PID du fils qui a été créé.

3. Les tubes anonymes

Dans cet exercice, nous utiliserons les tubes anonymes (pour rappel, ils sont créés à l'aide de pipe).

Prenez un soin particulier à fermer systématiquement tous les descripteurs des tubes non utilisés, aussi bien dans les fils que dans le père.

1. Écrivez un programme qui prend en argument un entier n et qui crée ensuite n tubes et n processus fils. Une fois tous les fils créés, le père lit depuis le clavier un numéro (entre 1 et n) et une chaîne de caractères. Il envoie ensuite la chaîne de caractères au fils spécifié qui l'affiche. Quand l'utilisateur saisit -1, le père envoie un code de fin aux fils qui s'arrêtent. Une fois tous les fils terminés, le père s'arrête.
2. Modifiez votre programme pour que chaque fils envoie la chaîne "X bien reçu" (où X est le numéro du processus) au père à chaque fois qu'il reçoit un message.

4. Les tubes nommés

Nous souhaitons reprendre l'exercice précédent mais en utilisant les tubes nommés. Nous avons besoin de deux programmes : le programme principal qui correspond au père de l'exercice précédent et le programme client qui correspond aux fils.

1. Le programme principal prend en argument un entier n et crée ensuite n tubes nommés dont le nom de fichier est "totoX" où X est un numéro entre 1 et n . Une fois qu'un client pour chaque tube est exécuté, le programme principal lit depuis le clavier un numéro (entre 1 et n) et une chaîne de caractères. Il envoie ensuite la chaîne de caractères au client associé qui l'affiche. Quand l'utilisateur saisit -1, le programme principal envoie un code de fin aux clients qui s'arrêtent.
2. Que se passe-t-il si deux clients sont exécutés sur le même tube ?
3. Modifiez vos programmes pour que chaque client envoie "X bien reçu" (où X est le numéro du client) au programme principal à chaque fois qu'il reçoit un message.

5. Concurrency

Selon M. Rabat, it is possible to have concurrency problems when several threads write in the same tube data of size greater than `TAILLE_BUF` (4096). As you do not believe what you see, you want to make sure.

The test program will create threads that will write in the same tube and the father will be content to read from this tube and verify that everything is correct.

The thread procedure takes 2 parameters: the number 'num' of the thread (included in $[0; N-1]$) and the descriptor to write in the tube. This procedure uses a static buffer of `SIZE` x char (where `SIZE` = 1000 at first). It initializes it with the character 'a' + num (the thread 0 will therefore fill the buffer with 'a', the second with 'b', etc.). The procedure then enters an infinite loop: it writes the buffer completely in the tube.

The father creates the tube and the threads. It then enters an infinite loop in which it reads a buffer from the tube. It ensures that all characters are identical (it is enough to compare each case of the buffer read to the first). It simply indicates if the buffer is different.

1. Write the two programs.
2. Execute the programs. What happens?
3. Change `SIZE` to 4097. What happens?
4. Finally, M. Rabat is right?